

CHAPTER

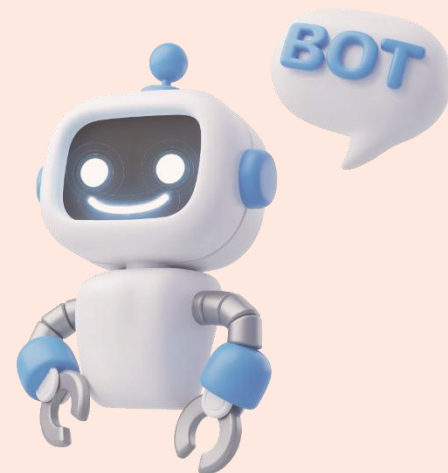
11

인공지능을 위한 기초 수학과 프로그래밍

Mathematics and Programming for AI

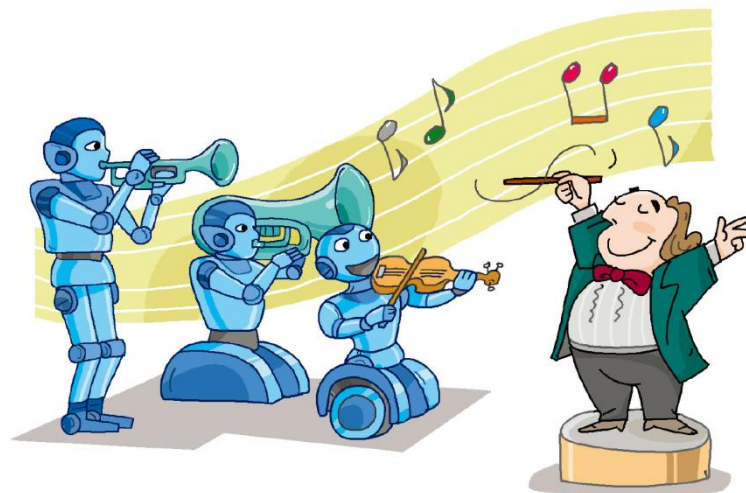
CONTENTS

- 11.1 인공지능을 위한 수학적 바탕
- 11.2 인공지능과 함수
- 11.3 미분과 델타 규칙
- 11.4 벡터의 개념과 표현
- 11.5 행렬과 행렬의 연산
- 11.6 확률, 통계, 회귀직선
- 11.7 인공지능용 프로그래밍 언어들

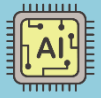


[단원의 주요 목표]

인공지능을 위한 수학적 배경
지식과 인공지능용 프로그래
밍 언어들을 고찰한다.



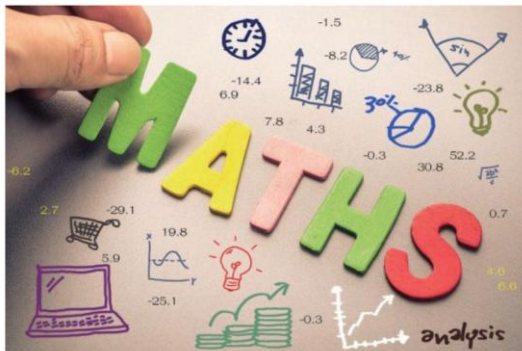
- 인공지능을 위한 여러 가지 기초적인 수학적 바탕을 살펴본다.
- 인공지능과 함수 그리고 미분과 델타 규칙을 알아본다.
- 벡터의 개념과 표현 그리고 행렬과 행렬의 연산을 다룬다.
- 확률, 통계, 그리고 회귀직선의 기초적인 사항들을 파악한다.
- 인공지능용 프로그래밍 언어들의 특징들을 알아본다.
- Python 다운로드를 단계별로 살펴보고 프로그램 실행 예를 살펴본다.



11.1 인공지능을 위한 수학적 바탕

(1) 인공지능과 수학

- 수준 높은 인공지능 탐구를 위해 수학적 바탕 필요
- 인공지능 관련 유명 인물 중 수학 관련 사람이 많음
- 예를 들어 앨런 튜링과 민스키(Minsky) 등도 수학자 출신
- 인공지능은 수학적 모델링과 관계가 깊음



[그림 11.1] 인공지능 탐구를 위한 수학적 바탕

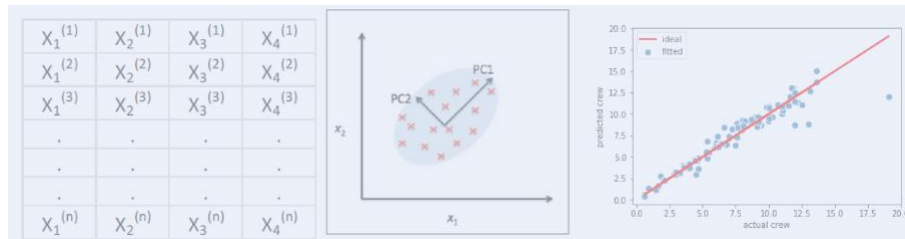


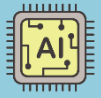


11.1 인공지능을 위한 수학적 바탕

(2) 인공지능 연구에 필요한 수학적 기초와 프로그래밍

- 인공지능에 관한 기본 지식은 현대사회의 중요 이슈
- 상식이나 교양수준에서는 간단한 인공지능의 개념, 간략한 원리, 응용 정도면 충분
- 인공지능 연구개발 예정자는 기초적인 수학 지식이 필요
- 행렬과 벡터는 매우 필요한 핵심적인 수학적 개념임
- 행렬은 데이터의 공간 변환 등에서 필수적인 도구

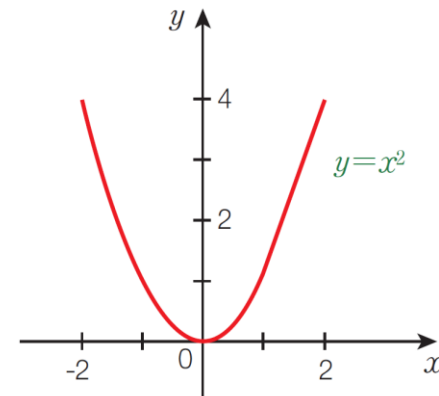
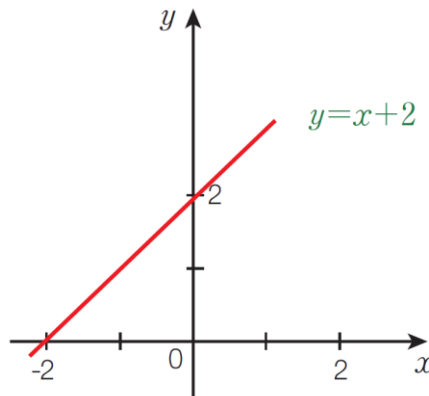




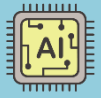
11.1 인공지능을 위한 수학적 바탕

인공지능에 필요한 주요 수학적 지식

- ① 고등학교에서 배운 기초 수학에 대한 지식이 필요
- 고등학교 때 배운 함수 관련 기초 지식 필요
 - 함수의 개념, 지수함수, 로그함수, 삼각함수,
 - 시그모이드 함수, 유클리드 기하 등



[그림 11.2] 기본적인 함수



11.1 인공지능을 위한 수학적 바탕

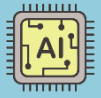
인공지능에 필요한 주요 수학적 지식(계속)

② 미분 개념 등이 중요

- 미분 개념은 인공지능의 최적화 과정에서 사용됨
- 미분은 신경망 학습에 필요한 기초 지식
- 델타 규칙과 역전파 알고리즘 등의 적용에 필요
- 신경망 학습에서 체인(chain) 규칙 사용

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx}$$

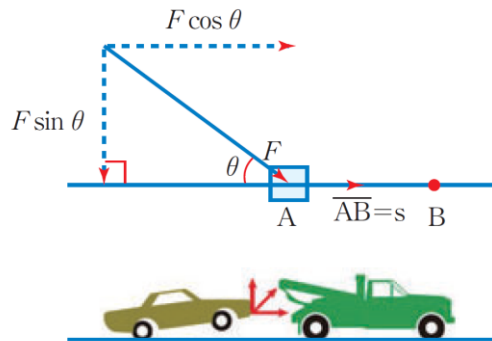
[그림 11.3] 체인 규칙



11.1 인공지능을 위한 수학적 바탕

인공지능에 필요한 주요 수학적 지식(계속)

- ③ 벡터(vector)와 관련된 기초 지식과 개념 이해가 중요
- 벡터는 신경망의 입력으로 들어갈 데이터 사용에 필요
 - 벡터의 내적과 벡터들 사이의 거리 측정 방법
 - 선형 변환 등의 지식이 필요



$$\begin{aligned} a_1 \mathbf{u} + a_2 \mathbf{v} + a_3 \mathbf{w} &= a_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} + a_2 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + a_3 \begin{bmatrix} 2 \\ 3 \end{bmatrix} \\ &= \begin{bmatrix} a_1 + a_2 + 2a_3 \\ a_1 + 3a_3 \end{bmatrix} \end{aligned}$$

[그림 11.4] 벡터의 개념과 벡터의 계산

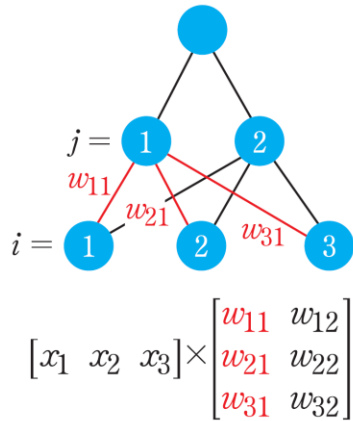


11.1 인공지능을 위한 수학적 바탕



인공지능에 필요한 주요 수학적 지식(계속)

- ④ 행렬(matrix)과 행렬식(determinant)에 관한 기본 지식이 요구됨
- 행렬의 곱셈과 선형 변환 등과 관련된 지식이 필요
 - 신경망에서는 행렬의 곱셈이 기본적으로 사용됨
 - 주어진 입력과 연결강도를 곱할 때의 행렬 연산에 필요



$$\begin{bmatrix} 6 & 5 \\ -4 & -3 \\ 7 & 6 \end{bmatrix} \begin{bmatrix} 2 \\ -3 \end{bmatrix} = \begin{bmatrix} 6 \cdot 2 + 5 \cdot (-3) \\ (-4) \cdot 2 + (-3) \cdot (-3) \\ 7 \cdot 2 + 6 \cdot (-3) \end{bmatrix} = \begin{bmatrix} -3 \\ 1 \\ -4 \end{bmatrix}$$

[그림 11.5] 신경망에서의 행렬의 곱셈

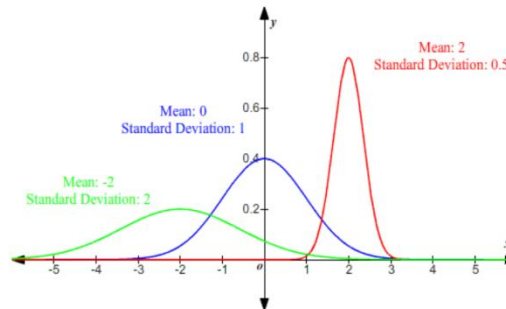


11.1 인공지능을 위한 수학적 바탕



인공지능에 필요한 주요 수학적 지식(계속)

- ⑤ 통계와 확률, 그리고 회귀 분석에 관한 기초 지식 필요
- 인공지능의 최종 결과물은 확률로 표시될 수도 있음
 - 통계에서 평균, 분산, 표준편차, 상관 계수 등 지식
 - 확률에 관한 지식
 - 머신러닝에서 분류를 위한 회귀분석에 대한 기초
 - 선형 회귀, K-mean, K-NN 분류 등의 기반 지식



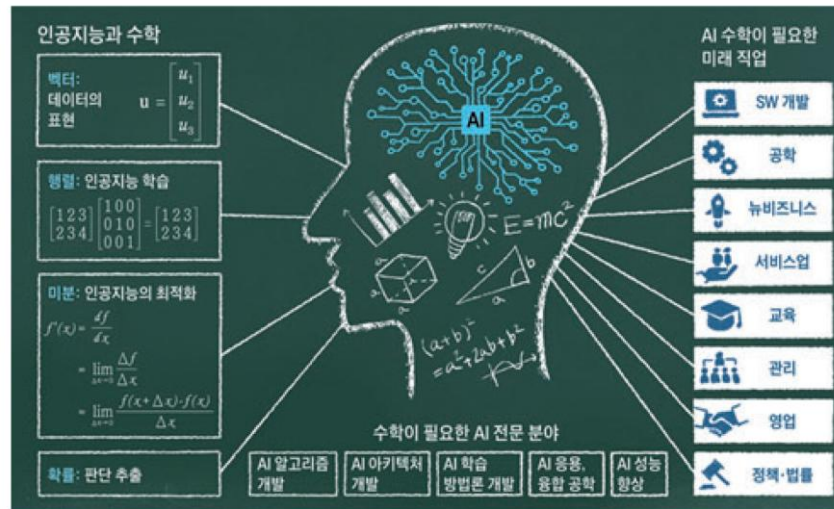


11.1 인공지능을 위한 수학적 바탕

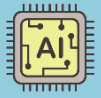


인공지능에 필요한 주요 수학적 지식(계속)

- ⑥ 신경망과 심층신경망의 구현을 위해 기초 지식이 필요
- 경사하강법, 역전파 알고리즘, 임계값, 선형 함수 등
 - 이산수학(discrete mathematics)에 관한 지식
 - 영상인식이나 음성인식을 위한 기본 지식



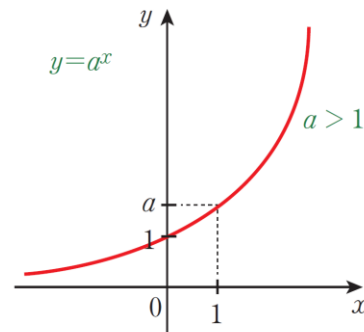
[그림 11.6] 인공지능과 수학과의 관계



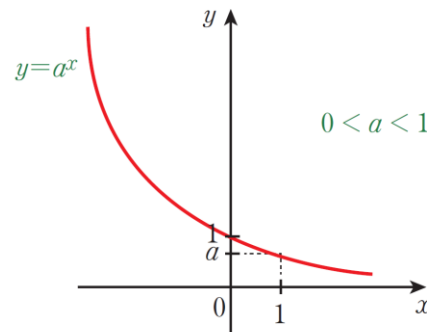
11.2 인공지능과 함수

(1) 지수함수(exponential function)

- 지수함수란 변수가 거듭제곱의 지수에 포함된 함수
- $y = a^x$, a 가 1이 아닌 양의 상수, x 가 모든 실수값일 때의 함수
- y 가 a 를 밑으로 하는 지수함수, a 의 값에 따라 2가지 경우
- $a > 1$ 일 경우 $y = a^x$ 는 증가함수, $0 < a < 1$ 일 경우 감소함수
- 두 경우 모두 그래프가 $y = 0$ 에 가까워지는 점근선 가짐

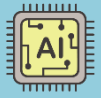


(1) a 가 1보다 클 때



(2) $0 < a < 1$ 일 때

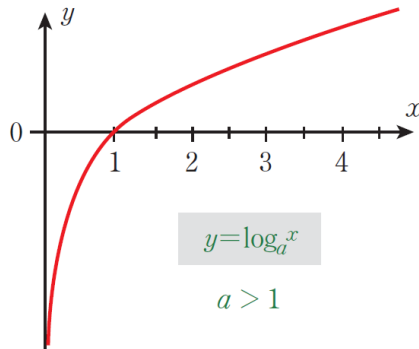
[그림 11.7] 지수함수



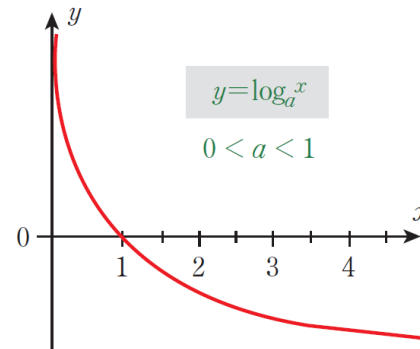
11.2 인공지능과 함수

(2) 로그함수

- $y = \log_a x$ ($a > 0, a \neq 1$)를 a 를 밑으로 하는 로그함수
- 지수함수 $y = a^x$ ($a > 0, a \neq 1$)의 역함수
- a 의 값에 따라 2가지로 구분되는데, 모두 $(1, 0)$ 점을 통과
- 두 경우 모두 그래프가 y 축에 가까워지는 점근선 가짐

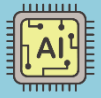


(1) a 가 1보다 클 때



(2) $0 < a < 1$ 일 때

[그림 11.8] 로그함수



11.2 인공지능과 함수

로그함수의 성질

- 로그함수는 다음과 같은 성질을 가지고 있음

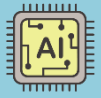
$$\log_a a = 1$$

$$\log_b (x^p) = p \log_b x$$

$$\log_b (xy) = \log_b x + \log_b y$$

$$\log_b \frac{x}{y} = \log_b x - \log_b y$$

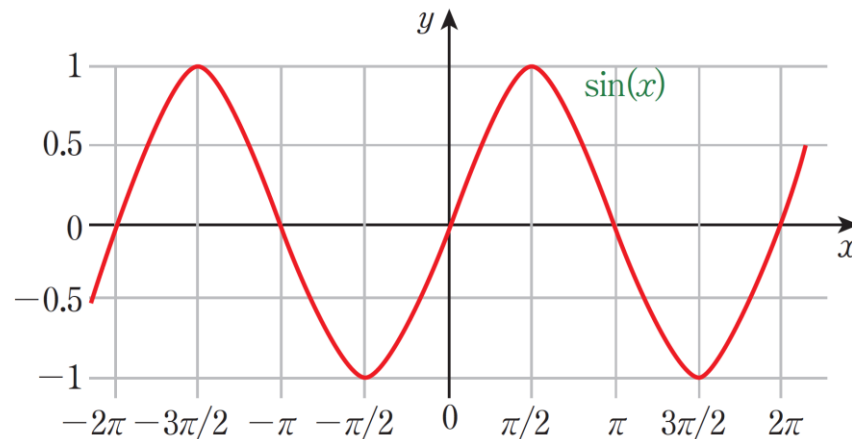
- 예를 들면 $\log_2 16 = 4$
- 그 이유는 $16 = 2 \times 2 \times 2 \times 2 = 2^4$ 이고 $\log_2 2 = 1$ 이기 때문
- 또 이와 같은 방법으로 $\log_2 \frac{1}{2} = -1$ 이 됨



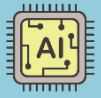
11.2 인공지능과 함수

(3) 삼각함수(Trigonometric functions)

- 삼각함수는 각의 크기에 따라 값이 달라지는 함수
- 대표적인 삼각함수로는 $\sin(x)$, $\cos(x)$, $\tan(x)$ 등
- 통상 180° 와 같은 각도 표현 대신 π 나 $\pi/2$ 와 같은 호도법 사용



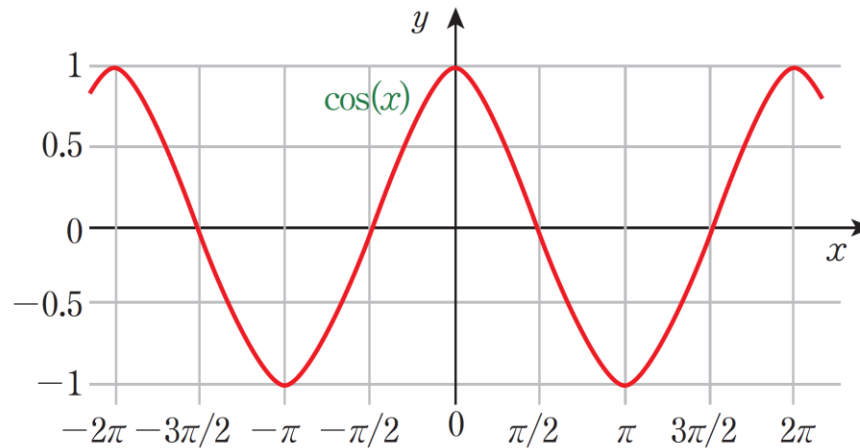
[그림 11.9] $y = \sin(x)$



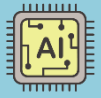
11.2 인공지능과 함수

삼각함수 - $\sin(x)$ 와 $\cos(x)$

- $y = \sin(x)$ 와 $y = \cos(x)$ 의 값은 각각 -1 에서 $+1$ 사이의 값을 가짐
- $\sin(x)$ 와 $\cos(x)$ 는 2π 를 주기로 같은 모양으로 순환
- 즉 $\sin(0) = \sin(2\pi) = 0$ 이고, $\cos(0) = \cos(2\pi) = 1$
- 또한 $\sin(\pi/2) = 1$ 이고 $\sin(\pi) = 0$, $\cos(\pi/2) = 0$ 이고 $\cos(\pi) = -1$



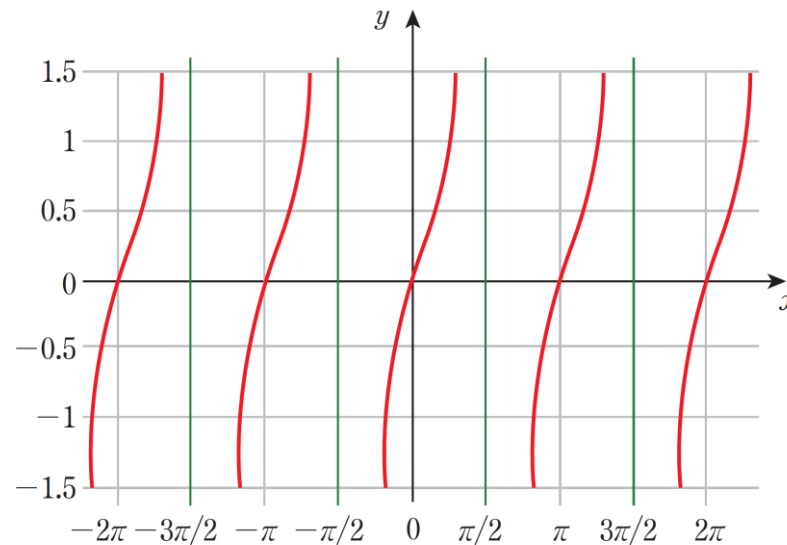
[그림 11.10] $y = \cos(x)$



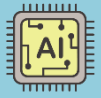
11.2 인공지능과 함수

삼각함수 - $\tan(x)$

- $y = \tan(x)$ 의 값은 [그림 6.11]에 나타난 바와 같음
- $y = \tan(x)$ 그래프는 π 를 주기로 같은 모양으로 순환
- $\tan(0) = 0$ 이고 $\tan(\pi) = 0$
- $\tan(\pi/2)$ 나 $\tan(-\pi/2)$ 등에서는 값이 정의되지 않음



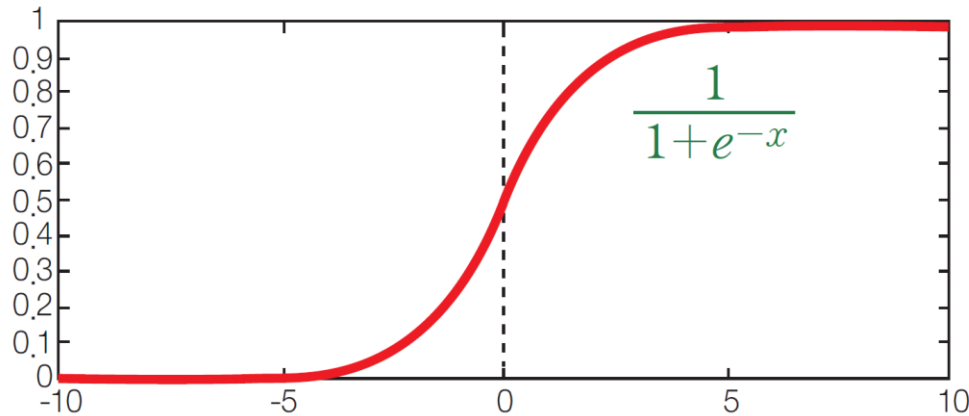
[그림 11.11] $y = \tan(x)$



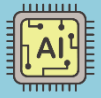
11.2 인공지능과 함수

(4) 대표적인 비선형 활성화 함수들

- 신경망에서 뉴런에 해당하는 노드는 비선형적(non-linear)임
- 특정한 활성화 함수(activation function)를 거쳐 출력을 냄
- 0과 1 사이의 값을 가지는 시그모이드(sigmoid) 함수가 많이 쓰임
- 시그모이드 함수는 신경망에서 출력을 결정할 때 많이 쓰임



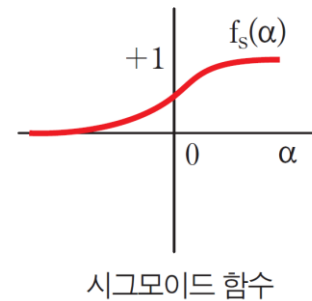
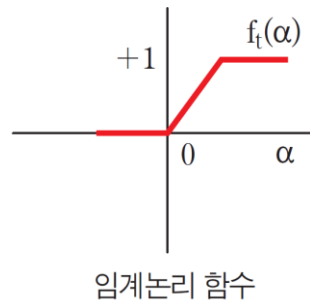
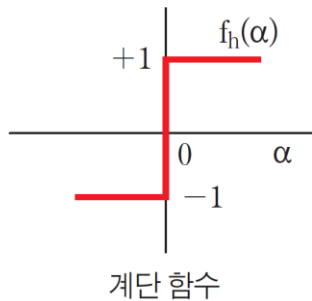
[그림 11.12] 시그모이드 함수의 식과 그래프



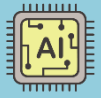
11.2 인공지능과 함수

비선형 활성화 함수들

- 그 외 활성화 함수로는 계단함수, 임계논리 함수 등이 많이 쓰임
- 계단함수에서는 x 축의 값이 음수일 때는 함수의 값이 모두 -1 이고, 양수인 경우에는 모두 $+1$ 이 됨
- 임계논리 함수에서는 x 축의 값이 음수일 때는 모두 0 이고, 0 과 1 사이에서는 $y = x$ 와 같은 선형의 값을, 그리고 1 이상에서는 모두 $+1$ 임



[그림 11.13] 3가지 대표적인 활성화 함수



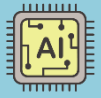
11.2 인공지능과 함수

(5) 손실 함수(loss function)

- 손실 함수란 예상한 값과 실제값과의 차이를 함수로 정의한 것
- 신경망에서의 손실 함수는 출력한 값과 실제값과의 오차에 대한 함수
- 신경망의 학습 과정은 이 오차를 최소로 줄이는 방향으로 진행
- 대표적인 손실 함수로는 평균제곱오차(MSE : Mean Squared Error)
- MSE는 예측값과 실제값 사이의 오차의 제곱에 대한 평균값

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

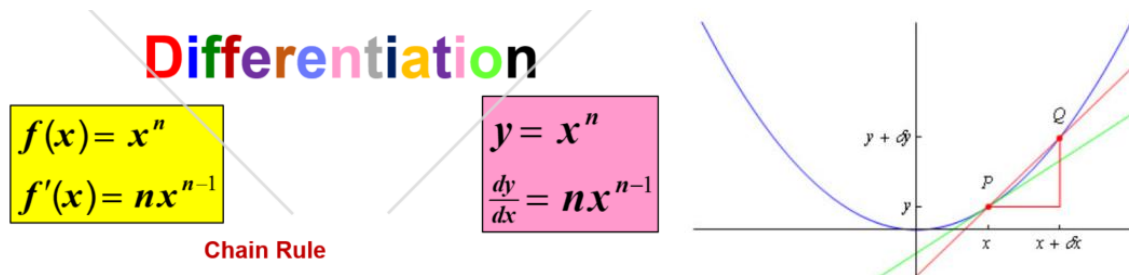
- 통상 MSE의 값이 작으면 추정된 값이 정답에 가까운 것이고, MSE의 값이 크면 정답과 멀리 떨어져 있다고 볼 수 있음
- 그 외의 손실 함수로는 교차 엔트로피 오차(CEE) 등의 방법

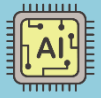


11.3 미분과 델타 규칙

(1) 미분(differentiation)과 도함수

- 미분이란 어떤 운동이나 함수의 순간적인 움직임을 서술하는 방법
- 어떤 함수의 미분이란 그것의 도함수를 도출해내는 과정
- 미분 공식으로 지수함수, 로그함수, 삼각함수 등의 도함수 구함
- 미분은 기하학적 관점에서 주어진 곡선의 접선을 구하는 문제와 같은 의미이며, 접선의 기하학적 의미는 곡선과 스치듯이 만나는 직선임





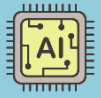
11.3 미분과 델타 규칙

(2) 도함수(derivative)의 정의

- 도함수란 정의역의 모든 x 에 대해 $f(x)$ 의 미분계수로 대응시키는 함수
- 기호로는 y' , $f'(x)$, dy/dx 로 나타내며 다음과 같이 도함수를 정의

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

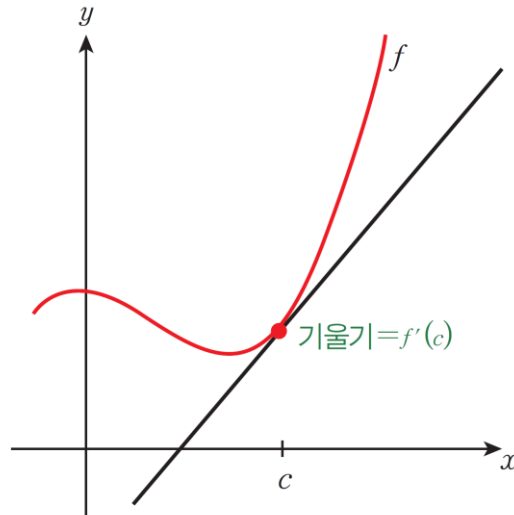
- $f(x)$ 의 도함수 $f'(x)$ 를 구하는 것이 미분이며, 그 계산법이 미분법
- 함수 f 가 입력값 x 에 따라 변할 때 각 입력값 x 에서 f 의 순간변화율을 구할 수 있는데, 이때 이 함수를 f 의 도함수라고 함



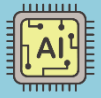
11.3 미분과 델타 규칙

순간변화율과 도함수

- 함수의 순간변화율은 함수의 각 점에서의 접선의 기울기를 의미
- 도함수는 입력값 x 마다 그 점에서 함수 그래프의 접선의 기울기를 대응시켜 주는 함수
- 그림에서 $x = c$ 에서의 도함수를 나타냄



[그림 11.14] $x = c$ 에서의 도함수



11.3 미분과 델타 규칙

(3) 미분의 체인 규칙(chain rule)

- 미분의 체인 규칙은 양파를 까는 것처럼 합성함수의 미분에 적용
- 바깥 함수를 먼저 미분한 후 다시 안쪽 함수를 미분하여 곱하는 방법
- 가령 y 를 x 에 관해 미분할 경우 먼저 y 를 u 에 관해 미분하고, 다시 u 를 x 에 관해 미분한 후 둘을 곱함
- 3개 이상의 변수에 대해서도 연쇄적으로 적용 가능

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx}$$

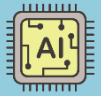
- 미분의 체인 규칙은 신경망에서 학습 이론을 유도하는 바탕이 됨



11.3 미분과 델타 규칙

(4) 델타 규칙(delta rule)과 경사하강법(gradient descent)

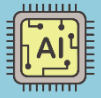
- 델타 규칙이란 단층 신경망 모델에서 사용되는 학습 방법 중 하나
- 실제 출력과 기대되는 출력 간의 차이를 최소화하기 위해 뉴런들 사이의 연결강도를 변화시키는데 쓰이는 방법
- 델타 규칙을 확장한 것이 일반화 델타 규칙(generalized delta rule)
- 1986년 Rumelhart에 의해 만들어진 다층 신경망을 학습시킬 수 있는 규칙으로, 오늘날 가장 유명해진 신경망 학습 규칙 중의 하나
- 델타 규칙에 의해 연결강도의 변화가 연결강도 공간상에 주어지는 오차의 제곱을 높이로 하는 곡면에 대해 경사하강법을 따르는데, 즉 오차의 제곱이 가장 많이 감소하는 방향으로 변화함



11.3 미분과 델타 규칙

(5) 인공지능과의 관련성

- 인공지능 중 신경망 관련 연구는 단층 퍼셉트론의 학습 기법에 바탕
- 신경망의 근본적인 이해가 필요한 사람은 수학적 지식 중 미분의 도함수 개념과 델타 규칙 및 체인 규칙에 대한 기초적인 이해가 필요
- 일반화 델타 규칙 관련 도함수 전개는 저자의 “신경망 이론과 응용(I)”편에 6페이지에 걸쳐 유도되어 있을 정도로 매우 복잡함
- 델타 규칙과 경사하강법은 신경망의 학습 규칙의 기반 이해에 필요
- 전문가 코스가 아닌 분은 수학적 지식에 너무 스트레스 받지 않기

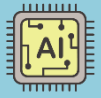


11.4 벡터의 개념과 표현

(1) 벡터의 정의

- 우리는 일상생활에서 여러 가지 측정값을 다루게 됨
- 스칼라(scalar)는 속도, 압력, 삼각형의 면적 등과 같은 물리적 양
- 벡터(vector)는 속도, 힘 그리고 가속도 등처럼 크기와 방향까지 포함
- 벡터는 통상 u , v , w 와 같이 굵은 글씨체의 소문자로 나타냄

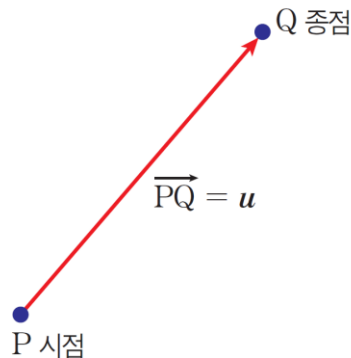
스칼라 → 크기
벡터 → 크기 + 방향



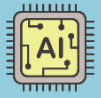
11.4 벡터의 개념과 표현

(2) 벡터의 개념과 표기법

- 벡터는 방향 정보를 가지고 있으므로 그래프를 이용해 표현 가능
- 일반적으로 벡터는 화살표(arrow)를 이용하여 표기
- 화살표는 벡터의 방향, 화살표의 길이는 벡터의 크기를 나타냄
- 화살표의 시작점인 P를 시점, 끝나는 점인 Q를 종점이라 함
- 벡터는 PQ 위의 화살표 표시나 볼드체 **a , b , u , v , ...** 등으로 표현



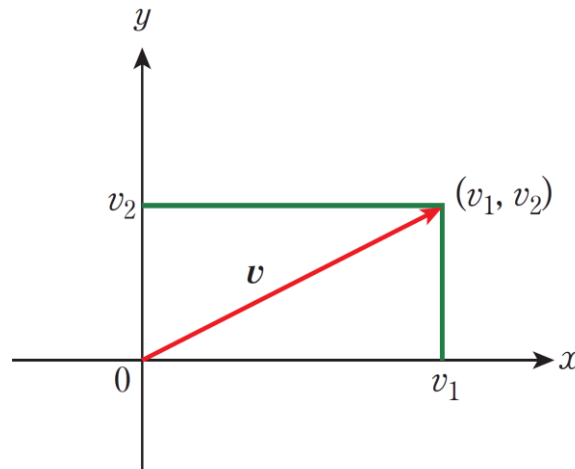
[그림 11.15] 시점과 종점이 있는 벡터



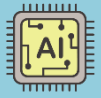
11.4 벡터의 개념과 표현

(3) 벡터의 크기와 기하학적 표현

- x, y 평면으로 표현되는 유클리드(Euclid)의 2차원 공간 R^2
- R^2 상에 있는 벡터 $\mathbf{v} = (v_1, v_2)$ 는 그림과 같이 원점을 시점으로 하여 종점 (v_1, v_2) 에 이르는 화살표로 표현
- 벡터의 크기는 화살표의 길이로 정의됨



[그림 11.16] R^2 상의 벡터 $\mathbf{v}$ 의 표현



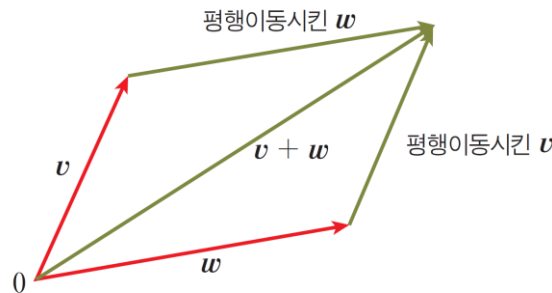
11.4 벡터의 개념과 표현

(4) 벡터의 연산

- $\mathbb{R}^n$ 상에서 $\mathbf{v} = (v_1, v_2, \dots, v_n)$ 와 $\mathbf{w} = (w_1, w_2, \dots, w_n)$ 가 주어진 경우
- 벡터의 합(sum)은 대응하는 각 성분들끼리 각각 더한 것

$$\mathbf{v} + \mathbf{w} = (v_1 + w_1, v_2 + w_2, \dots, v_n + w_n)$$

- $\mathbb{R}^n$ 상에서의 벡터 $\mathbf{v} + \mathbf{w}$ 의 표현은 [그림 6.17]과 같이 평행사변형 방향



[그림 11.17] $\mathbb{R}^n$ 상에서의 벡터 $\mathbf{v} + \mathbf{w}$ 의 표현



11.4 벡터의 개념과 표현

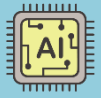
(5) 벡터의 내적(inner product, 內積)

- u, v 가 다음과 같은 R^n 상의 벡터인 경우

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{bmatrix}, \quad v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

- R^n 상의 벡터의 내적은 스칼라값으로 정의되며 $u \cdot v$ 로 나타냄

$$\begin{bmatrix} u_1 & u_2 & \cdots & u_n \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$



11.4 벡터의 개념과 표현

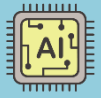
벡터 내적의 예

- 예를 들면, $u = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$ 와 $v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$ 가 $\mathbb{R}^2$ 상의 벡터인 경우
 $\mathbb{R}^2$ 상의 내적은 다음과 같이 정의되며 $u \cdot v$ 로 나타냄

$$u \cdot v = u_1 v_1 + u_2 v_2$$

- 또한 $u = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$ 와 $v = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}$ 가 $\mathbb{R}^3$ 상의 벡터인 경우
 $\mathbb{R}^3$ 상의 내적은 같은 맥락으로 $u \cdot v$ 로 나타내며 스칼라값을 가짐

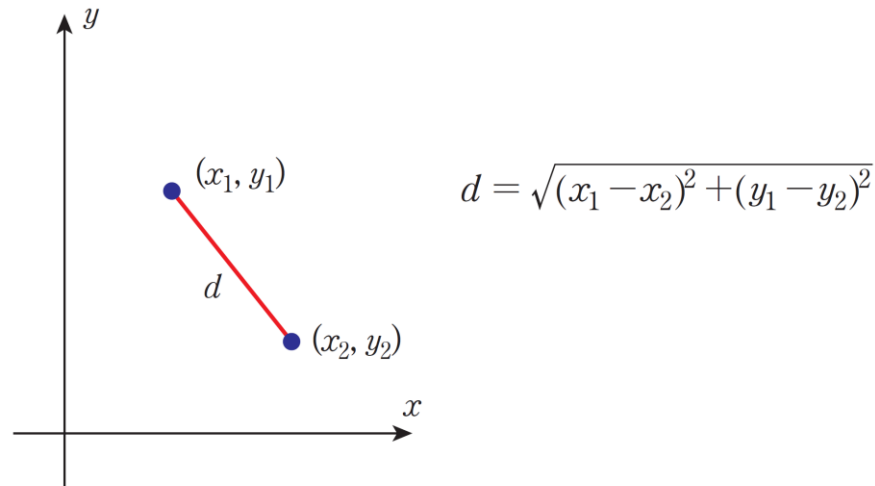
$$u \cdot v = u_1 v_1 + u_2 v_2 + u_3 v_3$$



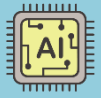
11.4 벡터의 개념과 표현

(6) 유클리드 거리(Euclidean distance)

- 유클리드 거리는 두 점 사이의 거리를 계산할 때 쓰는 방법
- 이 거리를 사용하여 유클리드 공간을 정의 가능
- 이 거리에 대응하는 노름(norm)을 유클리드 노름(Euclidean norm)이라 함



[그림 11.18] 두 점 사이의 거리



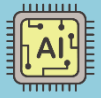
11.4 벡터의 개념과 표현

n차원 공간의 유클리드 거리

- 2차원 공간을 n차원 공간의 거리로 일반화하는 경우
- $\mathbf{P}(p_1, p_2, \dots, p_n)$ 와 $\mathbf{Q}(q_1, q_2, \dots, q_n)$ 사이의 거리는 다음과 같이 정의

$$\begin{aligned} & \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2 + \dots + (q_n - p_n)^2} \\ &= \sqrt{\sum_{i=1}^n (q_i - p_i)^2} \end{aligned}$$

- 이러한 거리 개념을 활용하여 신경망이나 머신러닝에서 K-mean, K-NN 등의 분류를 위한 클러스터링(clustering) 등에 활용 가능



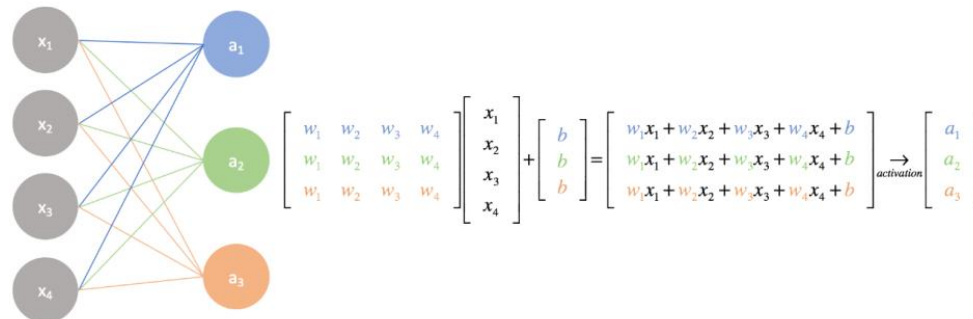
11.4 벡터의 개념과 표현

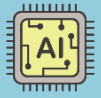
(7) 인공지능과의 관련성

- 신경망 학습에서 벡터 개념에 대한 기초적인 이해가 필요
- 뉴런으로 들어오는 입력과 연결강도의 곱은 벡터의 내적으로 연산됨
- 인공지능 관련 지식에는 문과와 이과의 구별이 없으므로 벡터에 대한 기초적인 지식이 필요
- 유클리드 거리는 각 점 사이의 거리를 측정함으로써 신경망의 분류(classification) 작업에 활용됨

Input layer Output layer

A simple neural network





11.5 행렬과 행렬의 연산

(1) 행렬(matrix, 行列)의 표현과 정의

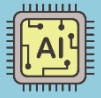
- 행렬은 수 또는 문자를 배열의 형태로 나타내는 것
- m, n 을 양의 정수라 할 때 다음과 같은 배열을 행렬이라 부름
- 행렬에서 행(row)은 가로 방향, 열(column)은 세로 방향을 의미

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

← 행

↑ 열

$m \times n$



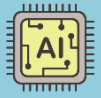
11.5 행렬과 행렬의 연산

행렬의 행과 열

- 앞의 행렬을 간단하게 $A = [a_{ij}]$, $i = 1, \dots, m$, $j = 1, \dots, n$ 이라 적음
- $m \times n$ 행렬 또는 (m, n) 행렬이라고 부름
- 이 행렬은 m 개의 행과 n 개의 열을 가지고 있음
- 예를 들면, 제1행은

$[a_{11}, a_{12}, \dots, a_{1n}]$ 이고,

제2열은 $\begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{m2} \end{bmatrix}$ 임

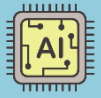


11.5 행렬과 행렬의 연산

행렬의 항(entry) 또는 성분(component)

- 행렬에서 a_{ij} 를 이 행렬의 ij -항(ij -entry)이라 부름
- 또는 a_{ij} 를 ij -성분(ij -component)이라고도 함
- a_{ij} 는 위로부터 i 번째의 행과 j 번째의 열이 만나는 항의 값

$$\begin{array}{c}
 j\text{번째 열} \\
 \downarrow \\
 A = \begin{bmatrix} a_{11} & \cdots & a_{1j} & \cdots & a_{1n} \\ \vdots & & \vdots & & \vdots \\ a_{i1} & \cdots & a_{ij} & \cdots & a_{in} \\ \vdots & & \vdots & & \vdots \\ a_{m1} & \cdots & a_{mj} & \cdots & a_{mn} \end{bmatrix} \leftarrow i\text{번째 행}
 \end{array}$$



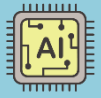
11.5 행렬과 행렬의 연산

행렬의 크기

- 다음은 행렬들의 크기를 나타냄
- 행의 개수 $\times$ 열의 개수의 형태로 표현

$$A = \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix} \text{는 } 3 \times 1 \text{ 행렬이고, } B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \text{는 } 3 \times 2 \text{ 행렬이며,}$$

$$C = \begin{bmatrix} 1 & -2 \\ 3 & -6 \\ 7 & -1 \\ 4 & 5 \end{bmatrix} \text{는 } 4 \times 2 \text{ 행렬이다.}$$



11.5 행렬과 행렬의 연산

행렬의 행과 열

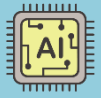
- 행렬의 크기 알아보기
- 행과 열의 벡터 형태의 표기 가능

예제 11.1 다음의 행렬 A 가 2×3 행렬임을 알아보자.

$$A = \begin{bmatrix} 1 & 1 & -2 \\ -1 & 4 & -5 \end{bmatrix}$$

풀이 이 행렬은 2개의 행과 3개의 열을 가진다. 행은 $[1, 1, -2]$, $[-1, 4, -5]$ 이고, 열은

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 \\ 4 \end{bmatrix}, \begin{bmatrix} -2 \\ -5 \end{bmatrix} \text{ 이다.}$$

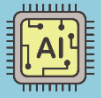


11.5 행렬과 행렬의 연산

행렬에서의 행벡터(row vector)와 열벡터(column vector)

- 행렬의 각 행은 가로의 n 순서쌍으로 볼 수 있음
- 행렬의 각 열은 세로의 m 순서쌍으로 볼 수 있음
- 가로의 n 순서쌍을 행벡터, 세로의 m 순서쌍을 열벡터라 부름
- 행벡터 $[x_1, \dots, x_n]$ 은 $1 \times n$ 행렬
- 다음과 같은 열벡터는 $m \times 1$ 행렬

$$\begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$



11.5 행렬과 행렬의 연산

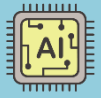
(2) 행렬의 합

- 행렬의 합은 그들이 같은 크기의 행렬일 때에만 정의됨
- 같은 크기의 행렬을 각각의 성분끼리 합하는 것

예제 11.2 $A = \begin{bmatrix} 3 & -1 \\ 2 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 2 & 1 \\ -3 & 2 \end{bmatrix}$ 일 때 $A+B$ 를 구해보자.

풀이

$$A+B = \begin{bmatrix} 3+2 & (-1)+1 \\ 2+(-3) & 4+2 \end{bmatrix} = \begin{bmatrix} 5 & 0 \\ -1 & 6 \end{bmatrix}$$



11.5 행렬과 행렬의 연산

(3) 행렬의 곱(multiplication)

- $A = [a_{ij}]$ 가 $m \times n$ 행렬이고, $B = [b_{ij}]$ 가 $n \times p$ 크기의 행렬인 경우
- 행렬 A와 B의 행렬의 곱은 $AB = C = [c_{ij}]$ 인 $m \times p$ 행렬

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}$$

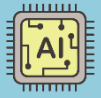
$$(i = 1, 2, \dots, m, j = 1, 2, \dots, p)$$

$$AB = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1j} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2j} & \cdots & b_{2p} \\ \vdots & \vdots & & \vdots & & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nj} & \cdots & b_{np} \end{bmatrix}$$

$m \times n$ $n \times p$

$$= \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{bmatrix} = C$$

$m \times p$



11.5 행렬과 행렬의 연산

행렬의 곱 연산

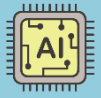
예제 11.3 행렬 A, B, C 가 다음과 같이 주어졌을 때, AB 와 AC 를 구해보자.

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 3 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & -2 \\ -1 & 2 \\ -2 & 4 \end{bmatrix}$$

풀이

$$\begin{aligned} (1) \quad AB &= \begin{bmatrix} 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} \text{row1} \times \text{col1} & \text{row1} \times \text{col2} \\ \text{row2} \times \text{col1} & \text{row2} \times \text{col2} \end{bmatrix} \\ &= \begin{bmatrix} (1)(2) + (1)(0) + (0)(1) & (1)(0) + (1)(1) + (0)(3) \\ (2)(2) + (0)(0) + (1)(1) & (2)(0) + (0)(1) + (1)(3) \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix} \end{aligned}$$

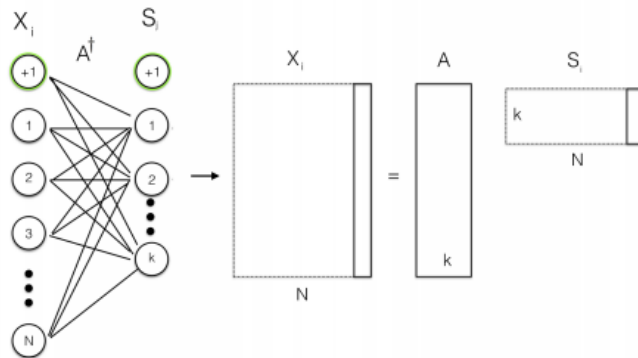
$$(2) \quad AC = \begin{bmatrix} 1 & 1 & 0 \\ 2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -2 \\ -1 & 2 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$



11.5 행렬과 행렬의 연산

(4) 인공지능과의 관련성

- 행렬은 신경망에서 벡터와 같은 형태의 연산을 매우 빠르게 처리 가능
- 신경망 학습에서 뉴런으로 들어오는 입력과 연결강도와의 곱 계산에 행렬의 곱이 필요
- 이와 같은 곱은 주로 벡터의 내적으로 연산됨
- 이 경우 행렬의 곱 연산을 이용하면 매우 편리
- 따라서 인공지능에서 행렬에 대한 기초적인 지식과 이해가 필요함





11.6 확률, 통계, 회귀직선

(1) 확률(Probability)

- 확률이란 어떤 사건 A 가 나타날 가능성을 수로 나타낸 것
- 사건 A 가 나타날 경우의 수를 전체 경우의 수로 나눈 값

[정의 11.1] 확률에 있어서 기본적인 법칙은 다음과 같다.

- ① 어떤 사건 A 에 대하여 $0 \leq P(A) \leq 1$
- ② 전사건 S 의 확률 $P(S) = 1$
- ③ 공사건 ϕ 의 확률 $P(\phi) = 0$
- ④ 사건 A 가 일어날 확률과 사건 A 의 여사건 A^c 가 일어날 확률 사이의 관계는 다음과 같다.

$$P(A) + P(A^c) = 1, \quad P(A^c) = 1 - P(A)$$



11.6 확률, 통계, 회귀직선

확률의 계산

- 확률을 구하는 예제들

예제 11.4 주사위 2개를 동시에 던져서 나온 수의 합이 3 또는 4가 될 확률을 구해보자.

풀이 수의 합이 3이 될 경우의 수는 (1, 2), (2, 1)이므로 확률은 $\frac{2}{36}$ 이고, 수의 합이 4가 될 경우의 수는 (1, 3), (2, 2), (3, 1)이므로 확률은 $\frac{3}{36}$ 이다. 따라서 확률은 $\frac{2}{36} + \frac{3}{36} = \frac{5}{36}$ 가 된다.

예제 11.5 주사위 2개를 동시에 던져서 나온 수의 합이 3 이상일 경우의 확률을 구해보자.

풀이 이 경우에는 여사건으로 문제를 푸는 것이 훨씬 간단하다. 수의 합이 3 미만인 경우, 즉 2인 경우는 $\frac{1}{36}$ 이다. 따라서 $1 - \frac{1}{36} = \frac{35}{36}$ 이다.



11.6 확률, 통계, 회귀직선

확률 변수, 이산적 확률, 확률 분포

[정의 11.2] 변수 X 가 취할 수 있는 모든 값이 $x_1, x_2, x_3, x_4, \dots, x_n$ 이고 X 가 이들 값을 취할 확률 $p_1, p_2, p_3, p_4, \dots, p_n$ 이 정해져 있을 때, 변수 X 를 확률 변수라고 한다. 또 확률 변수 X 가 이산적인 값을 취할 때 이러한 확률을 이산적 확률이라 한다.

예를 들어 한 개의 주사위를 던져서 나타나는 수를 X 라 하면 X 가 취할 수 있는 값은 1, 2, 3, 4, 5, 6이고 $P(x \leq 2)$ 는 X 가 1 또는 2의 값을 취하는 확률을 뜻한다.

[정의 11.3] 확률 변수 X 가 취하는 값 x_1 와 X 가 x_i 를 취할 확률 p_i 와의 대응 관계를 확률 변수 X 의 확률 분포라고 한다.



11.6 확률, 통계, 회귀직선

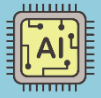
평균, 분산, 표준편차

[정의 11.4] 확률 변수 X 의 확률 분포가 다음과 같을 때, 평균(mean)이란 자료 전체의 합을 자료의 개수로 나눈 값을 말한다. 즉 $\sum_{i=1}^n x_i p_i = x_1 p_1 + x_2 p_2 + x_3 p_3 + x_4 p_4 + \cdots + x_n p_n$ 을 X 의 평균 또는 기대값(expectation)이라 하고, m 또는 $E(X)$ 로 나타낸다.

[정의 11.5] 확률 변수 X 의 평균이 m 일 때 $E((X-m)^2)$ 을 X 의 분산(variance, 分散)이라 하고, $V(X)$ 또는 $\sigma^2(X)$ 로 나타낸다.

$$V(X) = \sum_{i=1}^n (x_i - m)^2 P_i$$

[정의 11.6] 분산의 양의 제곱근을 표준편차(standard deviation)라 하고, $\sigma(X)$ 로 나타낸다. 표준편차 역시 평균으로부터 얼마나 퍼져 있는가를 알려주는 수치로 사용된다.



11.6 확률, 통계, 회귀직선

평균, 분산, 표준편차 계산의 예

예제 11.6 주사위를 하나 던질 때 나오는 숫자를 확률 변수 X 라고 할 때 X 의 평균, 분산, 표준편차를 각각 구해보자.

풀이 주사위 하나를 던질 때 경우의 수는 1, 2, 3, 4, 5, 6 인데, 확률 변수를 X 라 하면 X 는 1부터 6 사이의 값을 가지게 된다. 이에 대응하는 확률 분포는 모든 경우에 $\frac{1}{6}$ 이 된다.

따라서 평균 m 은

$$m = \sum_{i=1}^n x_i P_i = 1 \times \frac{1}{6} + 2 \times \frac{1}{6} + 3 \times \frac{1}{6} + 4 \times \frac{1}{6} + 5 \times \frac{1}{6} + 6 \times \frac{1}{6} = \frac{7}{2}$$

그리고 분산은

$$\begin{aligned} V(X) &= \sum_{i=1}^n (x_i - m)^2 P_i \\ &= \left\{ \left(1 - \frac{7}{2}\right)^2 + \left(2 - \frac{7}{2}\right)^2 + \left(3 - \frac{7}{2}\right)^2 + \left(4 - \frac{7}{2}\right)^2 + \left(5 - \frac{7}{2}\right)^2 + \left(6 - \frac{7}{2}\right)^2 \right\} \times \frac{1}{6} = \frac{35}{12} \end{aligned}$$

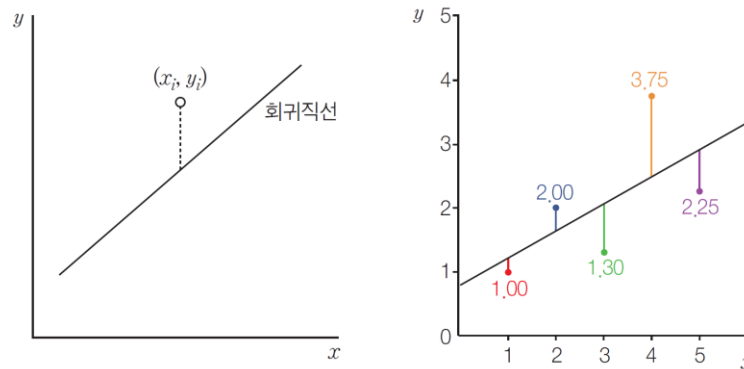
따라서 표준편차는 $\sigma(X) = \sqrt{\frac{35}{12}} = \sqrt{\frac{105}{6}}$ 이 된다.



11.6 확률, 통계, 회귀직선

(3) 회귀직선(regression line)

- 회귀직선이란 두 변수 X, Y 간의 관계를 나타내는 직선 $Y=a+bX$
- 각 점의 직선에서의 수직 거리의 제곱의 총합이 최소가 되게 a, b 결정
- 즉 점 (x_i, y_i) 에서 회귀직선까지의 y 축 방향의 거리 제곱의 총합을 최소로 해서 얻어지는 직선임
- 이때 직선 $y = a + bx$ 를 x 에 대한 y 의 회귀직선이라 함
- 여기서 a 는 y 절편이고 b 는 회귀직선의 기울기를 나타냄



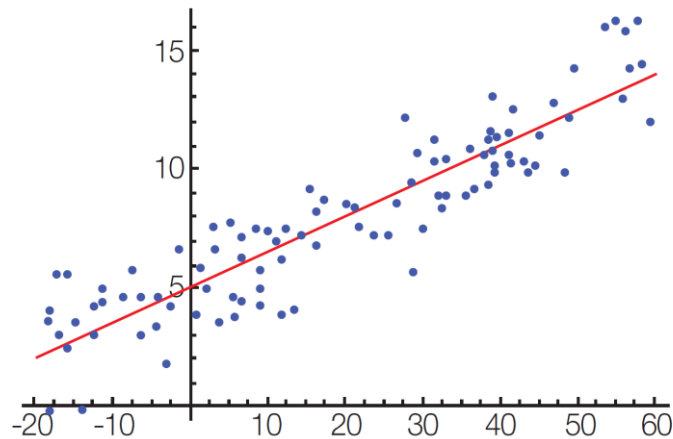
[그림 11.19] 회귀직선



11.6 확률, 통계, 회귀직선

회귀직선과 회기 분석(regression analysis)

- 그 결과 [그림 6.20]과 같은 완성된 회귀직선을 구할 수 있음
- 이를 바탕으로 8장에서 회귀 분석 가능
- K-mean, K-NN 분류 등의 기반 지식은 8장에서 다룸



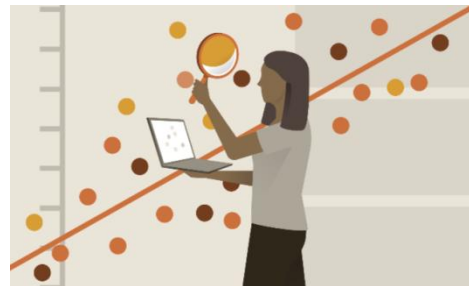
[그림 11.20] 완성된 회귀직선의 예



11.6 확률, 통계, 회귀직선

(4) 인공지능과의 관련성

- 인공지능 특히 신경망에서는 학습 결과가 확률적으로 나옴
- 그러므로 확률과 통계에 대한 기초적인 지식이 필요
- 회귀직선은 인공지능의 예측 기능과 직접적인 관련 있음
- 회귀직선을 이용한 회귀분석 기능을 인공지능과 접합
- 날씨나 경제적 지표에 대한 예측 등 활용도가 매우 큼





11.7 인공지능용 프로그래밍 언어들

(1) 인공지능용 프로그래밍 언어

- 프로그래밍 언어(Programming Languages: PL)
- 인간과 컴퓨터 사이의 의사소통용 인공적인 언어
- 인공지능의 초기 응용 분야에는 Lisp 많이 사용
- 논리를 처리하는데 편리한 Prolog도 많이 이용
- 신경망이나 딥러닝의 경우 Python이 많이 쓰임



[그림 11.21] 다양한 프로그래밍 언어들



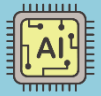
11.7 인공지능용 프로그래밍 언어들

(2) Lisp

- Lisp(리쉬으로 발음) 언어는 List processing의 약자
- 1960년 매카시에 의해 개발되어 지금도 활용되고 있음
- Lisp은 기호 형태의 자료형을 주로 쉽게 처리
- 리스트(list)와 트리(tree) 형태로 자료 구조 처리



[그림 11.22] Lisp 로고



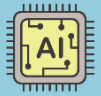
11.7 인공지능용 프로그래밍 언어들

Lisp의 주요 특징들 요약

- 대화식으로 구성된 인터프리터 방식의 언어
- 프로그램과 자료가 같은 형태로 취급됨
- 소프트웨어들이 Lisp으로 많이 개발되어 있어 활용도가 높음

(+ 3 4)	7	; 3 + 4 = 7
(PLUS 3 4)	7	; 3 + 4 = 7
(setq a 5)	5	; a에 5를 설정한다.
(car '(a b c))	a	; 리스트 중 가장 앞의 것을 리스트하라.
(cons 'a '(b c))	(a b c)	; 두 리스트를 결합하라.

[그림 11.23] Lisp의 간단한 예



11.7 인공지능용 프로그래밍 언어들

(3) Prolog

- Prolog(프롤로그)는 논리형 인공지능 언어
- 1972년 영국의 코왈스키(R. Kowalski)와 프랑스 연구진이 개발
- 논리에 기반을 둔 인공지능용 프로그래밍 언어
- 자연어 처리나 전문가 시스템 개발에 매우 유용
- 지식을 서술 논리로 표현하고 규칙(rule)에 따라 추론함



[그림 11.24] Prolog 프로그래밍 언어



11.7 인공지능용 프로그래밍 언어들



Prolog의 주요 특징 요약

- 사실(fact), 규칙(rule), 질문(question)들로 구성
- 인터프리터 언어이며 대화식의 명령 방식으로 작동
- 사실과 규칙들의 데이터베이스로 구성됨
- 질문에 응답하는 형식으로 진행
- 추론 엔진(inference engine)을 사용





11.7 인공지능용 프로그래밍 언어들

(4) Python

- 최근 Python이 인기 있는 프로그래밍 언어로 주목받기 시작
- 특히 코딩의 중요성이 강조되면서 사용자가 늘어나고 있음
- 문법이 비교적 간단하여 빠르고 쉽게 배울 수 있음
- 인터프리터 언어로서 실행 결과를 즉석에서 확인할 수 있음
- 플랫폼 독립적이며, 동적 타이핑 대화형 언어

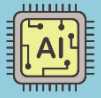


```

1  #!/usr/bin/env python
2  import sys
3  import os
4  import simpleknn
5  from bigfile import BigFile
6
7  if __name__ == "__main__":
8      trainCollection = 'toydata'
9      nimages = 2
10     feature = 'f1'
11     dim = 3
12
13     testCollection = trainCollection
14     testset = testCollection
15
16     featureDir = os.path.join(rootpath, trainCollection,
17                               'feature')
18     model = simpleknn.load_model(os.path.join(featureDir, 'model'))

```

[그림 11.25] Python 화면



11.7 인공지능용 프로그래밍 언어들

Python 언어

- Python은 고급 프로그래밍 언어의 일종
- 1991년 귀도 반 로섬(Guido van Rossum)이란 프로그래머가 발표
- Python 소프트웨어 재단이 관리하는 개방적인 무료의 프리웨어
- [그림 6.26]의 왼쪽은 절대값을 구하는 Python 프로그램의 예

Python

```
# absolute value  
  
n = int(argv[1])  
if n < 0:  
    abs = -n  
else:  
    abs = n  
print(abs)
```

Python script

```
1 def azureml_main(dataframe1 = None, data  
2 # Code to populate the result  
3     result = pandas.DataFrame(...)  
4     return result,  
5
```

[그림 11.26] Python 프로그램의 예



11.7 인공지능용 프로그래밍 언어들

Python 언어(계속)

- 프로그래밍 언어는 통상 다음과 같이 3가지 타입의 언어로 분류됨
- C와 같은 절차적 언어, Java와 같은 객체지향 언어, 그리고 Lisp와 같은 함수적 언어
- Python은 이 세 가지 특징들을 모두 가지고 있음



[그림 11.27] Python 개발자 귀도 반 로섬과 다운로드 화면

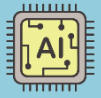


11.7 인공지능용 프로그래밍 언어들

Python의 주요 특징

- 인터프리터 언어, 실행 결과를 즉석에서 확인 가능
- 일반 프로그래밍 언어들보다 문법이 비교적 간단
- 함수의 정의는 함수형 언어의 형태를 취하고 있음
- 문장의 끝을 표시하는 세미콜론(;) 기호가 없음
- 들여쓰기를 사용하여 블록을 구분하는 문법 채용
- 플랫폼 독립적이며 동적 타이핑 대화형 언어
- C와 달리 초보자에게 어려운 포인터 개념 안 씀
- C언어로 구현된 C Python 버전이 사실상의 표준임





11.7 인공지능용 프로그래밍 언어들

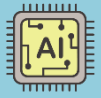
(5) R과 C

- R은 통계적 계산을 위한 오픈소스 소프트웨어
- 그래픽 시각화에 강점이 있는 프로그래밍 언어
- 인공지능의 머신러닝, 빅데이터, 데이터 사이언스에 활용
- 개발되어 있는 수많은 패키지 활용 가능, 기능 확장에 사용
- C는 범용 언어로서, 인공지능에서도 쓰이고 있음
- C는 Python 언어의 기반이 된 언어



```
> sum(2,7,5)
[1] 14
> x
[1] 2 NA 3 1 4
> sum(x)      # if any element is NA or NaN, result is NA or NaN
[1] NA
> sum(x, na.rm=TRUE)    # this way we can ignore NA and NaN values
[1] 10
> mean(x, na.rm=TRUE)
[1] 2.5
> prod(x, na.rm=TRUE)
[1] 24
```

[그림 11.28] R 로고와 합과 평균을 구하는 프로그램



11.7 인공지능용 프로그래밍 언어들

(6) 텐서플로와 파이토치

- 인공지능 프로그래밍 언어는 Python, Lisp, Prolog, R, C 등
- 딥러닝의 효율적인 실행을 위해 신경망 라이브러리 활용
- 텐서플로(TensorFlow)나 파이토치(PyTorch) 등
- 텐서플로와 파이토치는 9장에서 보다 자세하게 다루었음



[그림 11.29] 텐서플로와 파이토치